Information Retrieval

Part 1
Module Introduction and Indexing

Week 1

Qianni Zhang

Course Aims

- Knowledge of the underlying empirical and theoretical research issues relating to the indexing and retrieving of information objects.
- Understanding the criteria that make an information retrieval system efficient (fast) and effective (good).
- Skills necessary to design, implement and experiment with information retrieval systems.
- Indexing – Retrieval - Evaluation

Motivation and Applications

- There is too much unstructured “Data”
- Data is not the same as information

Data → Information → Knowledge

- Information is worthless if it cannot be found (extracted from raw data)
- Knowledge=Processed information retrieved from data

Today

- Key information
  - Coursework
  - Important Dates
  - Assessment
- First lecture:
  - Introduction to Information Retrieval (IR)
  - Indexing and TFIDF

Teaching schedule this year

- Lectures
  - 2 hours lecture every week, w1-w12
  - Our main teaching delivery
  - Recording will be uploaded after the lecture finishes

Thursday 10-12

- Labs
  - 2 hours every week, every week from w2 to w11

Monday 2-4pm

In-term assessment

Assignment 1 (15%), week 1-6:

- **Review of related work and a search engine design:**
  - Teams of 3-4
  - A single report
  - Introduction (motivation, design objectives, …)
  - Related work: a summary of related papers that inspire you in the design process
    - Starting references are provided, you search for further references.
    - Ideally identify the gap or difference between statements in the paper.
  - Search engine design
    - System structure
    - Details on components
    - Data, requirements, evaluation strategy
  - Workplan
  - Conclusion

In-term assessment

Assignment 2 (20%), week 7-11:

- **Develop a search engine:**
  - Same teams of 3-4, implement and evaluate the search engine.
  - Technology: Python, Lucene (Java APIs) and/or any other tool (Gate, Lemur, Terrier, …, Elasticsearch, Solr)
  - Submission of a presentation file, recording of presentation and demo, code, any data helps reproduce your experiments….
  - Evaluation on
    - Clarity
    - Technicality
    - Novelty

Assessment

- 35% coursework (group based)
  - 15%: Review of related work and research paper presentation: report document
  - 20%: Search engine presentation and demo: slides, source code package, video recording

  - Marking: group marks unless strong evidence for individual contributions; submission details see QM+.
  - Marking criteria: Clarity, technicality and novelty

- 65% final exam (online)

Important Dates

- Week 1-2:
  - Build your team of 3-4 students.
  - Deadline for building your group: **Thursday week 3**
  - Any student who has not chosen their group by then will be allocated to a group at random
- Week 6:
  - Submission of your review and search engine design.
  - *The labs and lectures enable you to develop the design.*
- Week 11:
  - Recording of presentation and demo of your search engine.
  - Submission of recording, code, and presentation file.

Module team

Lecturers and MO:

- Qianni Zhang (qianni.zhang@qmul.ac.uk) (MO)
  https://www.seresearch.qmul.ac.uk/cmai/people/qzhang/

- Yunfei Long (yunfei.long@qmul.ac.uk)
  https://www.seresearch.qmul.ac.uk/chcc/people/ylong/

Demonstrators: lab supervision, Q&A, assessment, …

- Chunchang Shao (c.shao@qmul.ac.uk)
- Yuhan Xia (yuhan.xia@qmul.ac.uk)
- Jinming Zhang (jinming.zhang@qmul.ac.uk)

What will we cover?

Classic IR Models

- Introduction + Indexing and TF-IDF
- Classic Retrieval Models I: Boolean, VSM, BM25, evaluation
- Classic Retrieval Models II: Probabilistic IR, LM, DFR
- Classic Retrieval Models III: Pagerank, Termsets, Fuzzy
- Classic Retrieval Models IV: others

Deep Information Retrieval

- LLMs in Information Retrieval
- LLM-Based Document Generation and Ranking
- Retrieval-Augmented Generation (RAG)
- Fine-Tuning LLMs for IR Components
- Evaluation, Applications, and Trends

Not necessarily in that order

Introduction to IR

Introduction to Information Retrieval (IR)

- Information Need
- Retrieval Tasks
- A Conceptual Model for IR
- Document and Document Representation
- Queries
- Best Match Retrieval

Database v.s. IR?

A (Simple) Database Example

Student Table

Student ID	Last Name	First Name	Department ID	email
1	Maryam	Karimzadehgar	CS	mkarimz2@qmul.ac.uk
2	Peters	jordan	EE	kj@qmul.ac.uk
3	Smith	Chris	EE	sc@qmul.ac.uk
4	Smith	John	CS	Sj@qmul.ac.uk

Department Table

Department ID	Department
EE	Electronic Engineering
CS	Computer Science

Course Table

Course ID	Course Description
cs736	Information Technology
ee750	Communication

Enrollment Table

Student ID	Course ID	Grades
1	cs736	90
1	ee750	75
2	cs736	95
2	ee750	80
3	cs736	60
4	ee750	77

Databases vs. IR

- Format of data:
  - DB: Structured data. Clear semantics based on a formal model.
  - IR: Mostly unstructured. Free text.
- Queries:
  - DB: Formal (like SQL)
  - IR: often expressed in natural language (keywords search)
- Result:
  - DB: exact result
  - IR: Sometimes relevant, often not

What is IR?

- Goal: Find the documents most relevant to a given Query

- Dealing with notions of:
  - Collection of documents
  - Query (User's information need)
  - Notion of Relevancy

Information Need

- Example of an information need in the context of the world wide web:

Find all documents (*information!*) about universities in the UK that

(1) offer master degrees in Information Retrieval and

(2) are registered with ACM SIGIR.

The information (*the document!*) should include full curriculum, fees, student campus, e-mail and other contact details.

- Formal representation of an information need = **Query**

Types of Information Needs

- Retrospective (Ad-hoc Querying)
  - “Searching the past”
  - Different queries posed against a static collection

- Prospective (Filtering)
  - “Searching the future”
  - Static query posed against a dynamic collection
  - Time dependent

Retrospective Searches

- Topical search

  Identify positive accomplishments of the Hubble telescope since it was launched in 1991.

  Compile a list of mammals that are considered to be endangered, identify their habitat and, if possible, specify what threatens them.

- Open-ended exploration

  Who makes the best chocolates?

  What technologies are available for digital reference desk services?

- Known-item search

  Find Qianni Zhang's homepage.

  What's the ISBN number of "Modern Information Retrieval"?

- Question answering

  "Factoid"

  Who discovered Oxygen?
  When did Hawaii become a state?
  Where is Ayer's Rock located?
  What team won the World Series in 1992?

  "Definition"

  Who is Aaron Copland?
  What is a quasar?

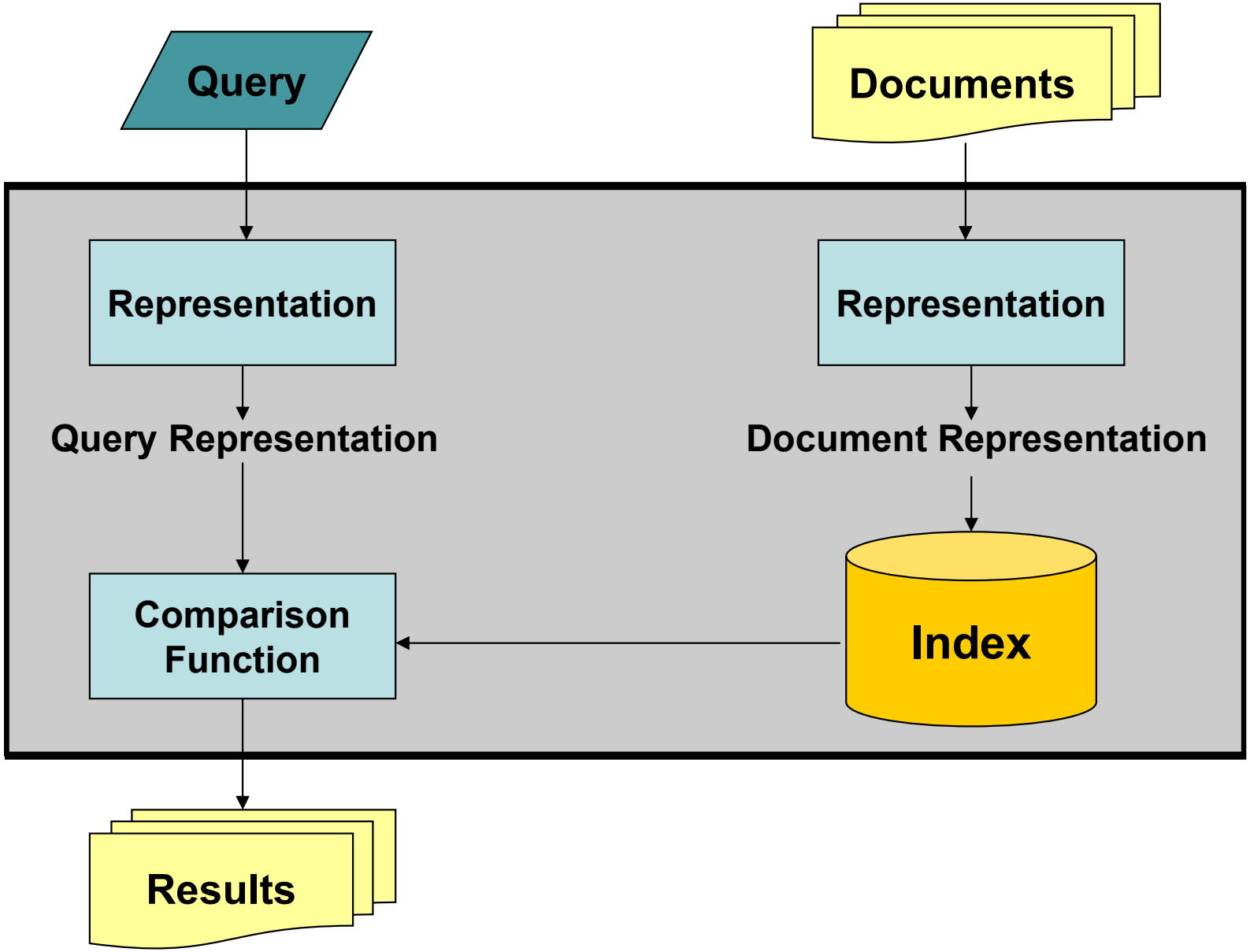
  "List"

  What countries export oil?
  Name U.S. cities that have a "Shubert" theater.

Prospective “Searches”

- Filtering
  - Make a binary decision about each incoming document
- Routing / Multi-label Classification
  - Sort incoming documents into different bins

Inside The IR Black Box



The Central Problem in IR

Information Seeker

Authors

Concepts

Concepts

Query Terms

Document Terms

Do these represent the same concepts?

Why IR is hard? Because Language is hard!!!

Ambiguity: e.g. Good Friday
Synonymy: e.g. classify/categorise
Polysemy: e.g. hamburger, bank
Morphology: e.g. sailing, sailor, sails
Paraphrase: different text, same meaning
Anaphora: e.g. he, she, it
Pragmatics: children vs grown-ups

Documents and Document Representations

Documents

- Unit of retrieval
- A passage of free text
  - composed of text, strings of characters from an alphabet
  - composed of natural language:

    newspaper articles, journal paper, dictionary definition, e-mail messages
  - size of documents:

    arbitrary, newspaper article vs journal article vs e-mail

- Sub-document can also be a unit of retrieval (passage, XML element, answer to a question)

Queries

- Information Need
- Simple queries
  - composed of two or three, perhaps of dozen of keywords
  - e.g. as in web retrieval
- Boolean queries
  - ‘neural network AND speech recognition’
  - e.g. as in online catalog and patent search
- Context queries
  - proximity search, phrase queries
  - e.g. ‘neural’ and ‘network’ distance at most 5 words

Best-Match Retrieval

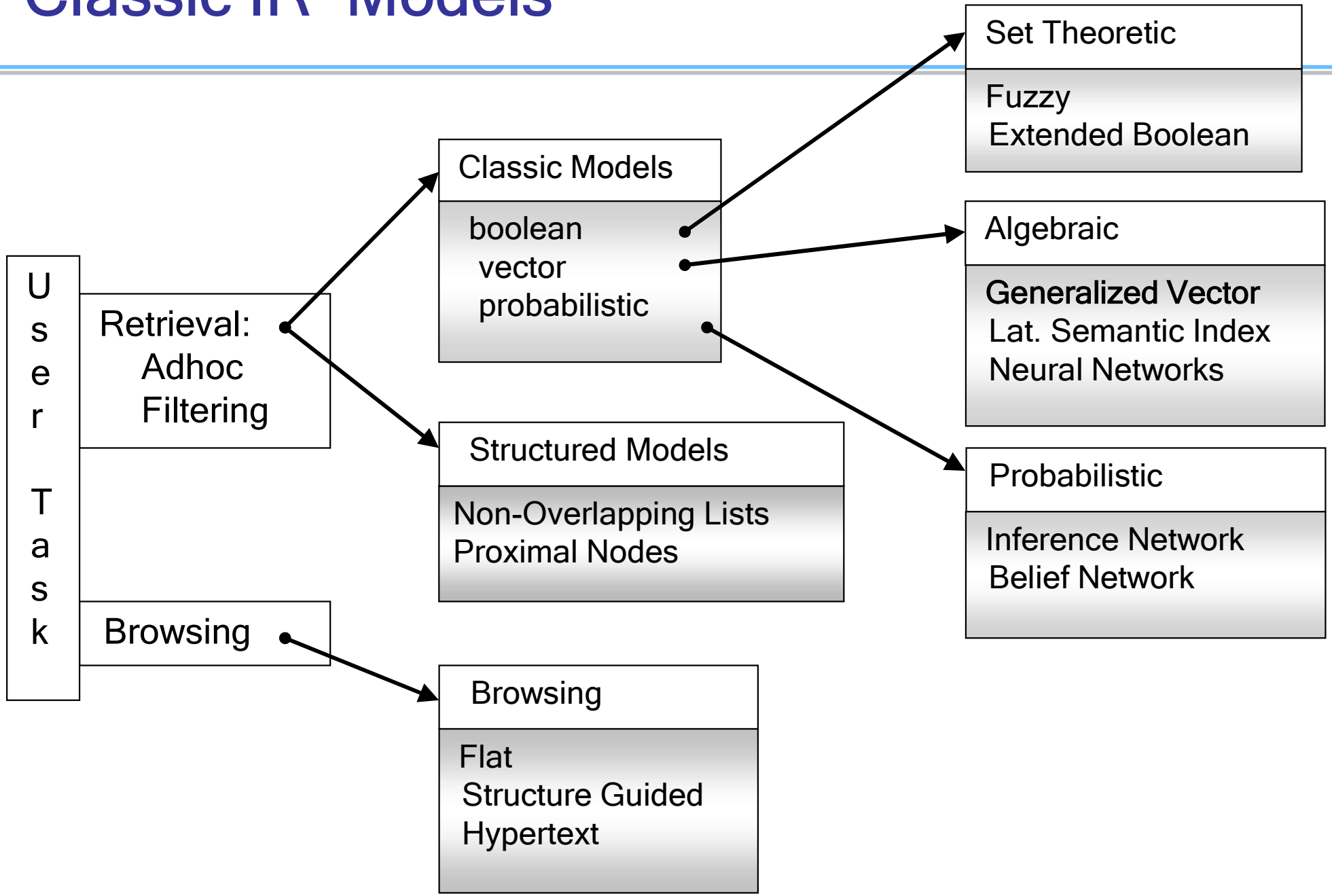
- Compare the terms in a document and query
- Compute “similarity” between each document in the collection and the query based on the terms they have in common
- Sorting the document in order of decreasing similarity with the query
- The outputs are a ranked list and displayed to the user – the top ones are more relevant as judged by the system

Document term descriptors to access text

$\longleftrightarrow$

User term descriptors characterising user needs

Classic IR Models



Search Output

- What now?
  - User identifies relevant documents for “delivery”
  - User issues new query based on content of result set
- What can the system do?
  - Assist the user to identify relevant documents
  - Assist the user to identify potentially useful query terms

Selection Interfaces

- One dimensional lists
  - What to display? title, source, date, summary, ratings, ...
  - What order to display? similarity score, date, alphabetic, ...
  - How much to display? number of hits
  - Other aids? related terms, suggested queries, …
- Two+ dimensional displays
  - Clustering, projection, contour maps, VR
  - Navigation: jump, pan, zoom

Query Expansion / Enrichment

- Relevance feedback
  - User designates “more like this” documents
  - System adds terms from those documents to the query
- Manual reformulation
  - Initial result set leads to better understanding of the problem domain
  - New query better approximates information need
- Automatic query suggestion

Evaluating IR Systems

- User-centered strategy
  - Recruit several users
  - Observe each user working with one or more retrieval systems
  - Measure which system works the “best”
- System-centered strategy
  - Given documents, queries, and relevance judgments
  - Try several variants of the retrieval method
  - Measure which variant is more effective

Indexing

Documents in IR

Document types

- **Monomedia document**: text document, etc.
- **Multimedia document**: document containing parts of different media
- **Hypertext document**: document with links; referred to as non-linear document
- **Hypermedia document**: multimedia + hypertext

- **User generated (content) document**: blogs, comments, tweets

Bag of Words

DOCUMENT

This is a document in information retrieval

INDEX

document
information
retrieval
is
this

Tokenization

- Treat text as sequence of discrete tokens or words
- Filter irrelevant words: Simplest approach is to ignore all numbers and punctuation and use only case-insensitive unbroken strings of alphabetic characters as tokens
- However, sometimes punctuation is meaningful. Examples: Emails, URLs
- More elaborated technique:
  - Analyse ? ! ; : “ ‘ [ ] ( ) < >
  - Extract question words: what? why? when?

Stopword Removal

- Very frequent words are not good discriminators

- list of stopwords
  - *a, about, above, according, across, after, afterwards, again, against, albeit, all, almost, alone, already, also, although, always, among, as, at*

- Stopwords are language dependent

- For efficiency, they are stored in a lookup table

Tokenization & stopwords (exercise)

Huge amounts of digital visual content are currently available in unstructured, non-indexed form. The availability of a wide range of digital recorders accessible to anyone, from cheap digital cameras to complex professional movie capturing devices, is making possible that the stocks of digital libraries already packed with visual content continue rising rapidly. This trend has generated an acute need for full automatic annotation systems able to index images at the same speed the population of digital libraries is growing. The lack of proper indexing and retrieval systems is rendering useless significant portions of available information. Actually, generating digital content is easy and cheap, managing and structuring it to produce effective services is not. This applies to the whole range of content owners, from professional digital libraries with their terabytes of visual content to the private collector of digital pictures stored in the disks of conventional personal computers.

(147 words)

Filtered result

huge amounts digital visual content are currently available unstructured nonindexed form availability wide range digital recorders accessible anyone cheap digital cameras complex professional movie capturing devices is making possible stocks digital libraries already packed visual content continue rising rapidly trend has generated acute need full automatic annotation systems able index images same speed population digital libraries is growing lack proper indexing retrieval systems is rendering useless significant portions available information actually generating digital content is easy cheap managing structuring produce effective services is this applies whole range content owners professional digital libraries terabytes visual content private collector digital pictures stored disks conventional personal computers

(104 words)

Lemmatization

- Reduce inflectional or variations of a word/verb to the base word
  - *am, are, is* → *be*
  - *dog, dogs, dog's* → *dog*
- *the dogs are playing in the garden* → *the dog is play in the garden*
- Reduce vocabulary size
- Generally improve recall but hurt precision
- A list of grammatical rules is needed
- A list of irregular words

Stemming

- Simplest: suffix stripping
  - *develop → develop*
  - *developing → develop*
  - *development → develop*
  - *developments → develop*

- It is language specific and can become complex.
- Stemming "blindly" strips off known affixes (prefixes and suffixes) in an iterative fashion.
  - *sses → ss*
  - *ies → i*
  - *ational → ate*
  - *tional → tion*

- The effectiveness of stemming:
  - For English: increase in recall doesn't compensate loss in precision

Statistical Properties of Text

Important questions:

- How is the frequency words in a language (English)?
- How fast does vocabulary size grow wrt. the size of the database?
- This can be used to select appropriate term weights and other strategies of IR systems.
- In English few words are very common, e.g. “the”, “of” can account for about 10% of words in a text
- Some words are very rare: half the words in a text appear only once
- “Heavy tailed” distribution, since most of the probability mass is in the “tail”

Bag of words

- Tokenization
- Stopwords removal
- Lemmatization
- Stemming

- What next?

Bag of words model

- Vector representations do not consider the ordering of words in a document
- Thus the documents
  - *John is quicker than Mary*
  - *Mary is quicker than John*
- have the same vectors
- are indistinguishable
- This is called the <u>bag of words</u> model.

Boolean Indexing

- Record whether a document contains a word: document is binary vector in {0,1}
- Idea: Query satisfaction = **overlap measure** = $|Q \cap D|$

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

- Formal definition of Boolean model will be given in the next lecture

Is the binary relationship {0,1} between a term and a document good enough?

TF-IDF

Term weighted indexing

Index Term Weighting

Effectiveness of an indexing language:

- Exhaustivity
  - number of different topics indexed
  - high exhaustivity: high recall and low precision
- Specificity
  - ability of the indexing language to describe topics precisely
  - high specificity: high precision and low recall

Term-document count matrices

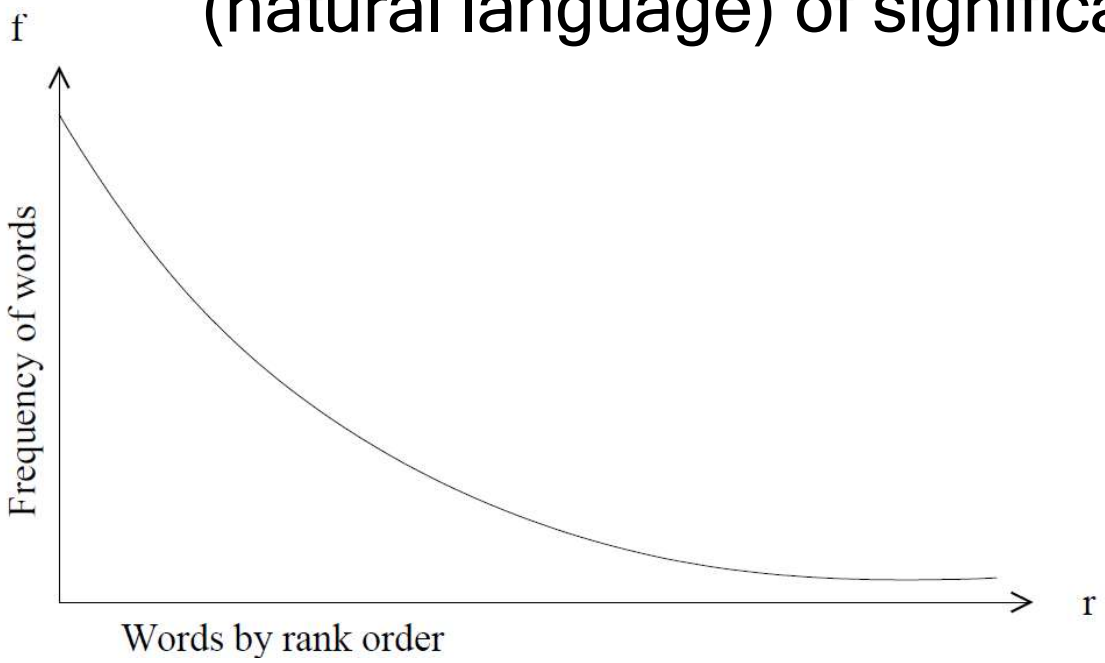
- Consider the number of occurrences of a term in a document:
  - Each document is a count vector in $\mathbb{N}^v$: a column below

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	0
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

Index Term Weighting

Zipf's law [1949]

Distribution of word frequencies is similar for different texts (natural language) of significantly large size



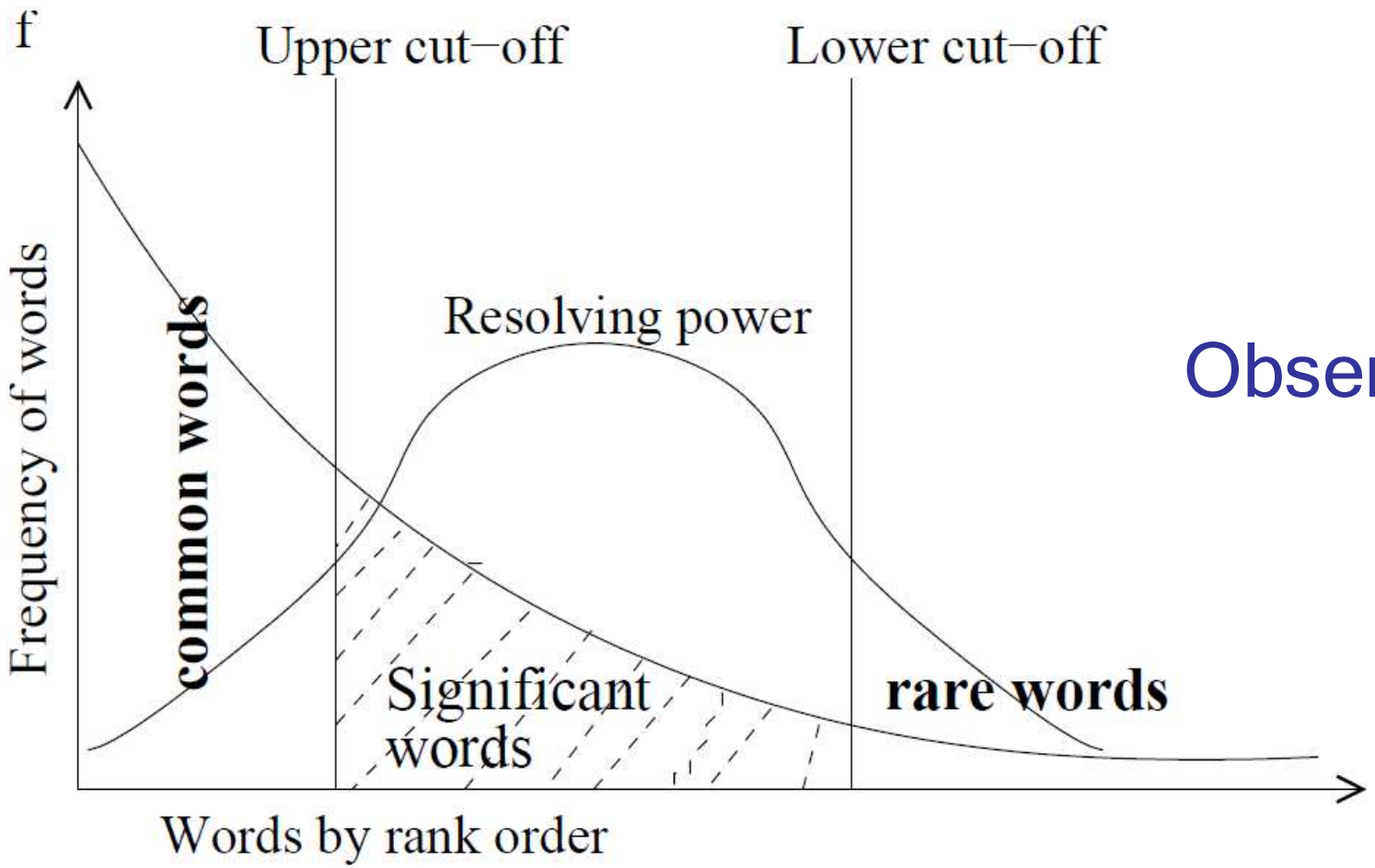
Given some corpus of natural language utterances, the frequency of any word is inversely proportional to its rank in the frequency table

- The most frequent word will occur approximately twice as often as the second most frequent word
- Example: "the" accounts for nearly 7% and "of" accounts for slightly over 3.5%
- Zipf's law holds for several languages

Index Term Weighting

Luhn's analysis – Hypothesis

Frequency of words is a measurement of word significance.
... a measurement of the power of a word to discriminate documents by their content ...



Observation

Method 1: weights terms using TF only

- Assign to each term a weight

  $tf_{t,d}$ - term frequency (how often term t occurs in document d)

$$score(q,d) = \sum_{t \in q} tf_{t,d}$$

- query = 'who wrote wild boys'
- doc1 = 'Duran Duran sang Wild Boys in 1984.'
- doc2 = 'Wild boys don't remain forever wild.'
- doc3 = 'Who brought wild flowers?'
- doc4 = 'It was John Krakauer who wrote *In to the wild.'*

query = {boys: 1, who: 1, wild: 1, wrote: 1}

doc1 = {1984: 1, boys: 1, duran: 2, in: 1, sang: 1, wild: 1}

doc2 = {boys: 1, don't: 1, forever: 1, remain: 1, wild: 2} ...

score(q, doc1) = 1 + 1 = 2 *score(q, doc2) = 1 + 2 = 3*

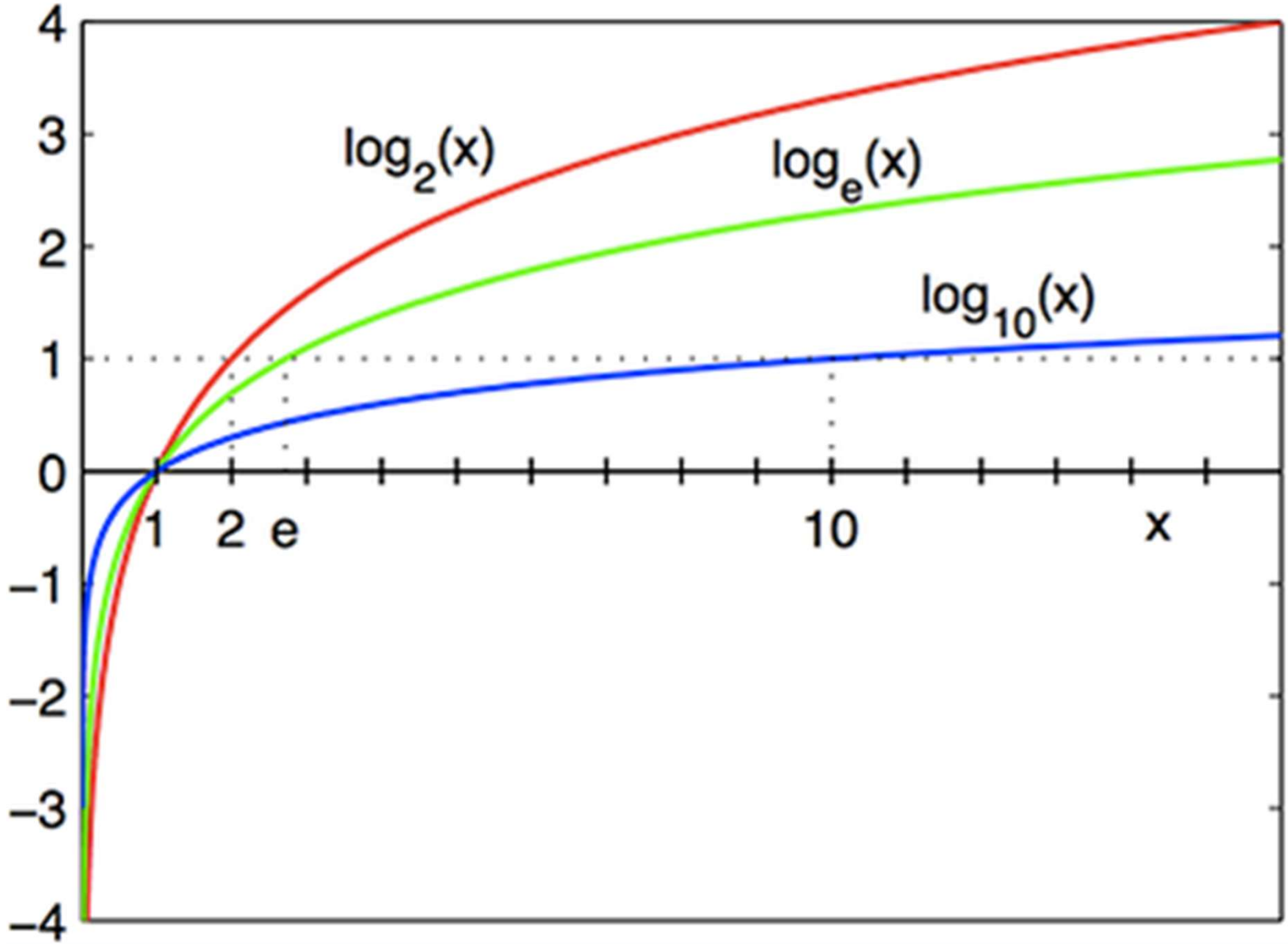
score(q,doc3) = 1 + 1 = 2 *score(q, doc4) = 1 + 1 + 1 = 3*

Term Frequency (TF)

- The term frequency $tf_{t,d}$ of term *t* in document *d* is defined as the number of times that *t* occurs in *d*.
- We want to use tf when computing query-document match scores. But how?
- Raw term frequency is not what we want:
  - A document with 10 occurrences of the term is more relevant than a document with 1 occurrence of the term.
  - But not 10 times more relevant.
- Relevance does not increase proportionally with term frequency.

NB: frequency = count in IR

What shall we do with TF?



Log-frequency weighting

- The log frequency weight of term t in d is

$$w_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{tf}_{t,d}, & \text{if } \mathrm{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

- 0 → 0, 1 → 1, 2 → 1.3, 10 → 2, 1000 → 4, etc.
- Score for a document-query pair: sum over terms *t* in both *q* and *d*:
- score $= \sum_{t \in q \cap d} (1 + \log \mathrm{tf}_{t,d})$

- The score is 0 if none of the query terms is present in the document.

Other forms of term frequency weighting

- Simplest form:

$$tf_{t,d} = f_{t,d} \text{ (raw count of a term in a document)}$$

- Boolean "frequencies":

$$tf_{t,d} = 1 \text{ if } t \text{ occurs in } d \text{ and 0 otherwise;}$$

- Term frequency adjusted for document length :

$$tf_{t,d} = f_{t,d} \div (number\ of\ words\ in\ d)$$

- Logarithmically scaled frequency:

$$tf_{t,d} = \begin{cases} 1 + \log(f_{t,d}), when\ f_{t,d} \neq 0 \\ 0, when\ f_{t,d} = 0 \end{cases}$$

- Augmented frequency:

$$tf_{t,d} = 0.5 + 0.5 \times \frac{f_{t,d}}{\max\{f_{t',d}: t' \in d\}}$$

Why is Method 1 (TF-only) not good?

- All terms have equal importance.
- Bigger documents have more terms, thus the score is larger.
- It ignores term order.

Postulate: If a word appears in every document, probably it is not that important (it has no discriminatory power).

Method 2: New weights: IDF

- Rare terms are more informative than frequent terms
  - Recall stop words
- Consider a term in the query that is rare in the collection (e.g., *arachnocentric*)
- A document containing this term is very likely to be relevant to the query *arachnocentric*
- → We want a high weight for rare terms like *arachnocentric*.

Method 2: New weights: IDF

IDF weight

- df_t is the <u>document frequency</u> of t: the number of documents that contain t
  - df_t is an inverse measure of the informativeness of t
  - $df_t \leq N$, where N is the number of documents in collection
- We define the idf (inverse document frequency) of t by

$$\mathrm{idf}_t = \log_{10}(N/\mathrm{df}_t)$$

- We use log (N/ df_t) instead of N/ df_t to "dampen" the effect of idf.
- Will turn out the base of the log is immaterial.

Effect of IDF on ranking

- Does idf have an effect on ranking for one-term queries, like
  - iPhone?
  - Why not?
- idf has no effect on ranking one term queries
  - idf affects the ranking of documents for queries with at least two terms
  - For the query capricious person, idf weighting makes occurrences of capricious count for much more in the final document ranking than occurrences of person.

TF-IDF

- df_t - document frequency for term t
- idf_t - inverse document frequency for term t

$$idf_t = \log \frac{N}{df_t}$$

N - total number of documents

- $tf\text{-}idf_{td}$ - a combined weight for term t in document d

$$tf\text{-}idf_{t,d} = tf_{t,d} * idf_t$$

$$score(q,d) = \sum_{t \in q} tf\ idf_{t,d}$$

- Increases with the number of occurrences *within* a doc
- Increases with the rarity of the term *across* the whole corpus
- tf-idf became popular as a weighting scheme in the vector-space model (VSM), SMART retrieval system, Salton [1971]

Example: idf values

terms	df	idf
1984	1	0.602
boys	2	0.301
brought	1	0.602
don’t	1	0.602
duran	1	0.602
flowers	1	0.602
forever	1	0.602
in	2	0.301
it	1	0.602
john	1	0.602

terms	df	idf
krakauer	1	0.602
remain	1	0.602
sang	1	0.602
the	1	0.602
to	1	0.602
was	1	0.602
who	2	0.301
wild	4	0.0
wrote	1	0.602

D1	duran duran sang wild boys in 1984
D2	wild boys don't remain forever wild
D3	who brought wild flowers
D4	it was john krakauer who wrote in to the wild

Example: calculating scores (1)

query = ‘who wrote wild boys’

documents	S: tf-idf	S: tf
duran duran sang wild boys in 1984	0.301	2
wild boys don't remain forever wild	0.301	3
who brought wild flowers	0.301	2
it was john krakauer who wrote in to the wild	**0.903**	**3**

terms	df	idf
who	2	0.301
wild	4	0.0
wrote	1	0.602
boys	2	0.301

Example: calculating scores (1)

query = ‘who wrote wild boys’

documents	**S: tf-idf**	**S: tf**
duran duran sang wild boys in 1984	0.426	2
wild boys don't remain forever wild	0.551	3
who brought ~~wild~~ flowers	0.301	1
it was john krakauer who wrote in to the wild	**1.028**	**3**

terms	df	idf
who	2	0.301
wild	**3**	**0.125**
wrote	1	0.602
boys	2	0.301

Example: calculating scores (2)

terms	df	idf
who	2	0.301
wild	**3**	**0.125**
wrote	1	0.602
boys	2	0.301

Original values

query = ‘who wrote wild boys’

documents	**S: tf-idf**	**S: tf**
duran duran who sang wild boys in 1984	0.551	3
wild boys don't remain forever wild	0.551	3
who brought ~~wild~~ flowers	0.125	1
it was john krakauer who wrote in to the wild	**0.852**	**3**

terms	df	idf
who	**3**	**0.125**
wild	**3**	**0.125**
wrote	1	0.602
boys	2	0.301

documents	**S: tf-idf**	**S: tf**
duran duran ~~sang~~ wrote wild boys in 1984	**0.727**	**3**
wild boys don't remain forever wild	0.551	3
who brought ~~wild~~ flowers	0.301	1
it was john krakauer who wrote in to the wild	**0.727**	**3**

terms	df	idf
who	2	0.301
wild	**3**	**0.125**
wrote	**2**	**0.301**
boys	2	0.301

Binary → count → weight matrix

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95

Each document is now represented by a real-valued vector of tf-idf weights $\in \mathbb{R}^{|V|}$

Documents as vectors: Vector Space Model

- So we have a |V|-dimensional vector space
- Terms are axes of the space
- Documents are points or vectors in this space
- Very high-dimensional: tens of millions of dimensions when you apply this to a web search engine
- These are very sparse vectors - most entries are zero.

Summary

- Bag-of-words Document Representation
- Document representation processing
- Boolean Indexing
- Term Weighting Theories (Zipf, Luhn)
- Weighted Index (TF, IDF)
- Other term weighting techniques